CHIA NẶNG NHẸ TRÊN CÂY

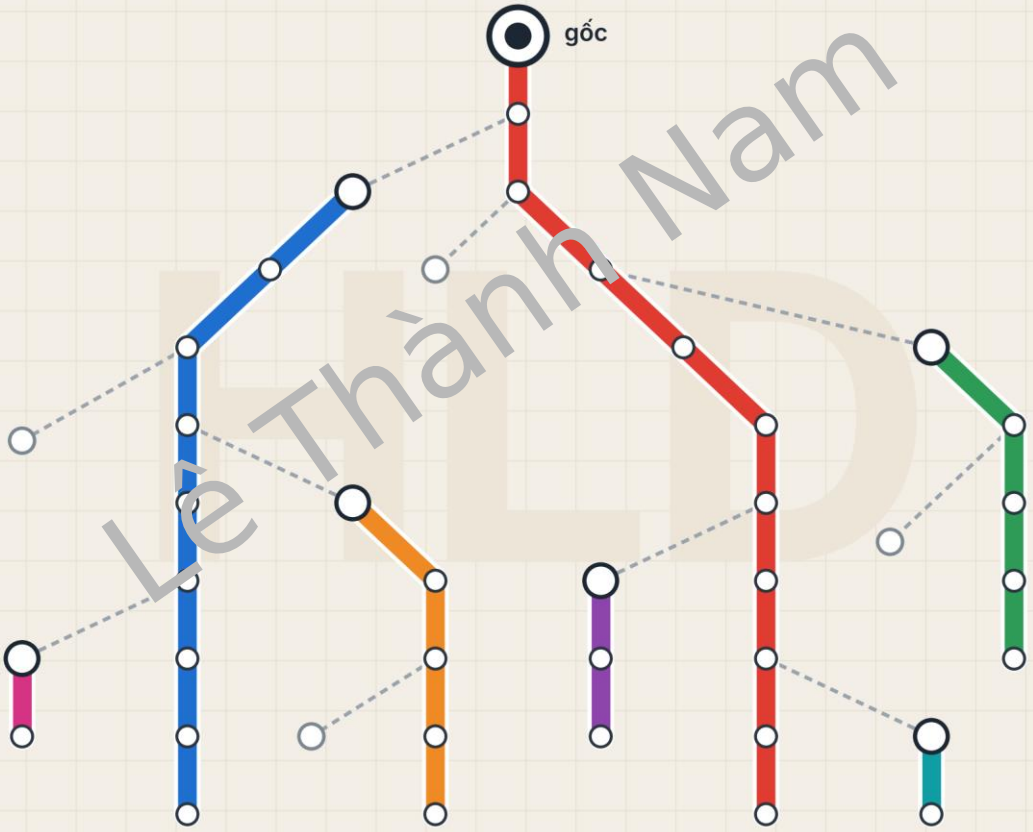
Heavy-Light Decomposition

Từ cơ bản đến nâng cao

3 **TẬP 3**

HƯỚNG DẪN GIẢI

`pos[head[u]] .. pos[u]`



Lê Thành Nam

chuỗi nặng: một tuyến liên tục

cạnh nhẹ: điểm đổi tuyến

Mọi hành trình về gốc: ≤ $\log_2 n$ lần đổi tuyến

Tổng hợp và biên soạn

Lê Thành Nam

LỜI NÓI ĐẦU

Cuốn sách này là Tập 3 của bộ tài liệu Chia nặng nhẹ trên cây (HLD): Từ cơ bản đến nâng cao, gồm hướng dẫn giải cho toàn bộ 36 bài tập của Tập 2. Mỗi lời giải được viết theo cùng một trình tự:

- Tóm tắt đề: Phát biểu lại bài toán bằng ngôn ngữ của cây và dãy số, gạt bỏ phần "câu chuyện".
- Phân tích: Các nhận xét then chốt, có lập luận và chạy tay trên ví dụ.
- Từng subtask: Cách làm đủ cho subtask đó, đếm số phép tính để thấy vì sao nó đủ nhanh và vì sao nó không đủ cho subtask sau, kèm chương trình hoàn chỉnh hoặc đoạn code ý chính.
- Bảng điểm: Mỗi cách làm đạt những subtask nào, được bao nhiêu điểm.
- Bẫy thường gặp: Những lỗi dễ mắc trong bài.

Mọi chương trình trong sách đều đã được biên dịch theo chuẩn C++14 và được đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên: cây ngẫu nhiên, đường thẳng, hình sao, hình răng lược, hình chổi, cây nhị phân đầy đủ, với số đỉnh cỡ nhỏ bị xáo trộn và cạnh cho theo thứ tự ngẫu nhiên. Các chương trình dành cho subtask đặc biệt được kiểm tra trên dữ liệu thỏa mãn đúng ràng buộc của subtask đó. Các chương trình tối ưu đều đã được chạy thử trên dữ liệu lớn nhất để bảo đảm thời gian.

Một lời khuyên: Đừng đọc lời giải như đọc truyện. Hãy đọc phần phân tích tới nhận xét đầu tiên, rồi gấp sách lại và tự làm tiếp. Chỉ mở lại khi đã thật sự bế tắc.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁC ĐOẠN CODE DÙNG CHUNG

Nhiều lời giải dùng lại cùng một vài đoạn code. Để không lặp lại lời giải thích, ta ghi chúng ở đây; mỗi chương trình trong sách vẫn được in **đầy đủ** để em chép và chạy được ngay.

Hàm build() chuẩn (Tập 1, Bài 5)

Tính p, dep (BFS từ gốc 1), sz, heavy (lượt ngược), rồi head, pos, id (ngăn xếp các đầu chuỗi, đi con nặng trước). Sau hàm này: Mỗi chuỗi là một đoạn liên tiếp trên dãy pos, cây con gốc u là đoạn [pos[u], pos[u] + sz[u] − 1], và id[pos[u]] = u.

Code ý chính: Hàm build(), cây cho dạng danh sách kề g, gốc 1

```
void build() {
    p.assign(n + 1, 0);
    dep.assign(n + 1, 0);
    sz.assign(n + 1, 1);
    heavy.assign(n + 1, 0);
    head.assign(n + 1, 0);
    pos.assign(n + 1, 0);
    id.assign(n + 1, 0);
    vector<int> order;
    order.push_back(1);
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : g[u])
            if (v != p[u]) {
                p[v] = u;
                dep[v] = dep[u] + 1;
                order.push_back(v);
            }
    }
    for (int i = n - 1; i > 0; i--) sz[p[order[i]]] += sz[order[i]];
    for (int u = 1; u <= n; u++)
        for (int v : g[u]) {
            if (v == p[u]) continue;
            if (heavy[u] == 0 || sz[v] > sz[heavy[u]]) heavy[u] = v;
        }
    int cur = 0;
    vector<int> st;
    st.push_back(1);
    while (!st.empty()) {
        int h = st.back();
        st.pop_back();
        for (int u = h; u != 0; u = heavy[u]) {
            head[u] = h;
            pos[u] = ++cur;
            id[cur] = u;
            for (int j = (int)g[u].size() - 1; j >= 0; j--) {
                int v = g[u][j];
                if (v != p[u] && v != heavy[u]) st.push_back(v);
            }
        }
    }
}
```

Khuôn mẫu chia nặng nhẹ (Tập 1, Bài 6)

Lê Thành Nam

Code ý chính: Xử lý mọi đoạn của đường đi từ u tới v (process là phần xử lý một đoạn)

```
while (head[u] != head[v]) {
    if (dep[head[u]] < dep[head[v]]) swap(u, v);
    process(pos[head[u]], pos[u]);
    u = p[head[u]];
}
if (dep[u] > dep[v]) swap(u, v);
process(pos[u], pos[v]);
```

Với giá trị đặt trên cạnh (Bài 7), đoạn cuối là [pos[u] + 1, pos[v]] và bỏ qua khi u = v. Khi phép gộp phụ thuộc thứ tự (Bài 9, 10), không được đổi chỗ u, v: Dùng hai biến tích lũy cho hai phía.

Đi lên từng bước để liệt kê đường đi (dùng cho các subtask nhỏ)

Code ý chính: Các đỉnh của đường đi từ u tới v theo đúng thứ tự

```
vector<int> path(int u, int v) {
    vector<int> L, R;
    while (dep[u] > dep[v]) {
        L.push_back(u);
        u = p[u];
    }
    while (dep[v] > dep[u]) {
        R.push_back(v);
        v = p[v];
    }
    while (u != v) {
        L.push back(u);
        R.push_back(v);
        u = p[u];
        v = p[v];
    }
    L.push back(u);
    for (int i = (int)R.size() - 1; i >= 0; i--) L.push back(R[i]);
    return L;
}
```

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. CHUẨN BỊ: ĐƯA CÂY VỀ DÃY SỐ 6
Bài 1.1. NHANVIEN: Quỹ lương phòng ban 7
Bài 1.2. CATCAY: Chia đôi mạng lưới 10
Bài 1.3. TONGKC: Địa điểm họp mặt 13
Bài 2.1. CAYCON: Ngân sách chi nhánh 17
Bài 2.2. THUONG: Thưởng Tết 22
Bài 2.3. THANGCHUC: Cấp dưới xuất sắc 27
Bài 3.1. MAXDOAN: Đỉnh cao nhất 31
Bài 3.2. GANDOAN: Sơn lại hàng rào 34
Bài 3.3. DOANCON: Chuỗi ngày lãi nhất 38

CHƯƠNG 2. CHIA NẶNG NHẸ 43
Bài 4.1. CHUOINANG: Các tuyến tàu 44
Bài 4.2. HANHKHACH: Hành khách đổi tàu 47
Bài 4.3. TONGDOI: Tổng số lần đổi tàu 51
Bài 5.1. LCAHLD: Tổ tiên chung 55
Bài 5.2. TOTIENK: Truy tìm cội nguồn 59
Bài 5.3. DINHK: Trạm dừng thứ k 63
Bài 6.1. TONGDUONG: Phí cầu đường 67
Bài 6.2. KIEMTRA: Kiểm tra an toàn 73
Bài 6.3. DEMNHO: Đếm cây thấp 80

CHƯƠNG 3. ỨNG DỤNG CỦA CHIA NẶNG NHẸ 87
Bài 7.1. CAUYEU: Cây cầu yếu nhất 88
Bài 7.2. PHIDUONG: Phí bảo trì 94
Bài 7.3. DUPHONG: Cáp dự phòng 102
Bài 8.1. TUOINUOC: Tưới nước 108
Bài 8.2. CANDAT: Trình quản lí gói 115
Bài 8.3. TONGLCA: Tổng độ sâu tổ tiên chung 121
Bài 9.1. DOIMAU: Đèn lồng 128
Bài 9.2. TANGDAN: Đường đi tăng dần 135
Bài 9.3. DOANLOI: Hành trình sinh lời 143
Bài 10.1. TOTIENLON: Tổ tiên đủ mạnh 149
Bài 10.2. NGANSACH: Ngân sách hành hương 155
Bài 10.3. HANHTRINH: Hành trình có giới hạn 161

CHƯƠNG 4. CHIA NẶNG NHẸ NÂNG CAO 168
Bài 11.1. GOCMOI: Thủ đô mới 169
Bài 11.2. QUOCGIA: Quốc gia xa xôi 174
Bài 11.3. GOCDONG: Gốc di động 180
Bài 12.1. MAUTROI: Màu áo trội nhất 187
Bài 12.2. XEPCHU: Xếp chữ đối xứng 192
Bài 12.3. TENRIENG: Những cái tên khác nhau 198

Lê Thành Nam

CHƯƠNG 1. CHUẨN BỊ: ĐƯA CÂY VỀ DÃY SỐ

Chương này hướng dẫn giải chín bài tập đi cùng Bài 1, 2, 3 của Tập 1. Các bài đều có lời giải O(n) hoặc O((n + q) log n) ngắn gọn; cái khó nằm ở việc nhìn ra đại lượng cần tính, và ở những chi tiết cài đặt nhỏ (thứ tự duyệt, kiểu dữ liệu, giá trị trung hòa). Hãy để ý cách các subtask được thiết kế: Subtask giữa thường là một "gợi ý" dẫn tới lời giải đầy đủ.

TRONG CHƯƠNG NÀY

Bài 1.1. NHANVIEN · **Bài 1.2.** CATCAY · **Bài 1.3.** TONGKC

Bài 2.1. CAYCON · **Bài 2.2.** THUONG · **Bài 2.3.** THANGCHUC

Bài 3.1. MAXDOAN · **Bài 3.2.** GANDOAN · **Bài 3.3.** DOANCON

Lê Thành Nam

BÀI 1.1 NHANVIEN · Quỹ lương phòng ban Cơ bản

TÓM TẮT ĐỀ

Cho cây gốc 1 dạng mảng cha và giá trị a[v]. Với mỗi đỉnh v, in tổng và giá trị lớn nhất của a trên cây con gốc v.

Phân tích

Đây là Mở rộng 1 của Bài 1: Tổng và max đều là phép toán **gộp được**, nên giá trị của cây con gốc u tính được từ giá trị của các cây con gốc con của u: s[u] = a[u] + tổng s[c], mx[u] = max(a[u], max mx[c]) với mọi con c của u.

Ba subtask ứng với ba cách tổ chức phép tính.

Subtask 1 Mỗi người báo cáo lên mọi cấp trên (60 điểm)

Mỗi đỉnh v đi lên từ v tới gốc, cộng a[v] vào s và cập nhật mx của mọi tổ tiên. Cách này không cần thứ tự duyệt nào.

NHANVIEN.CPP (C1): Đi lên từ mỗi đỉnh, O(n · h)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NHANVIEN.INP", "r")) {
        freopen("NHANVIEN.INP", "r", stdin);
        freopen("NHANVIEN.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<long long> a(n + 1), s(n + 1, 0), mx(n + 1, 0);
    vector<int> p(n + 1, 0);
    for (int v = 1; v <= n; v++) cin >> a[v];
    for (int v = 2; v <= n; v++) cin >> p[v];
    for (int v = 1; v <= n; v++)
        for (int x = v; x != 0; x = p[x]) {
            s[x] += a[v];
            mx[x] = max(mx[x], a[v]);
        }
    for (int v = 1; v <= n; v++) cout << s[v] << ' ' << mx[v] << '\n';
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Đỉnh v đi qua dep[v] + 1 tổ tiên. Tổng số bước là tổng (dep[v] + 1) ≤ n·(h + 1). Subtask 1: không quá 10^6 bước. Subtask 2: h ≤ 1000 nên không quá $2 \cdot 10^5 \cdot 1001 \approx 2 \cdot 10^8$ phép cộng đơn giản, chạy khoảng 0,3 giây. Subtask 3: cây đường thẳng cho n(n + 1)/2 ≈ $2 \cdot 10^{10}$ bước, quá chậm.

Lê Thành Nam

Subtask 2 Gộp từ dưới lên theo thứ tự BFS (100 điểm)

Lập thứ tự BFS từ gốc (cha đứng trước con), rồi đi ngược thứ tự, mỗi đỉnh v gửi s[v] và mx[v] lên cha. Khi v gửi đi, mọi con của v (đứng sau v) đã gửi xong, nên s[v], mx[v] đã đầy đủ.

NHANVIEN.CPP: Hai lượt BFS, O(n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("NHANVIEN.INP", "r")) {
        freopen("NHANVIEN.INP", "r", stdin);
        freopen("NHANVIEN.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<long long> a(n + 1), s(n + 1), mx(n + 1);
    vector<int> p(n + 1, 0);
    vector<vector<int>> g(n + 1);
    for (int v = 1; v <= n; v++) cin >> a[v];
    for (int v = 2; v <= n; v++) {
        cin >> p[v];
        g[p[v]].push_back(v);
    }
    vector<int> order;
    order.push_back(1);
    for (int i = 0; i < (int)order.size(); i++)
        for (int v : g[order[i]]) order.push_back(v);
    for (int v = 1; v <= n; v++) s[v] = mx[v] = a[v];
    for (int i = n - 1; i > 0; i--) {
        int v = order[i];
        s[p[v]] += s[v];
        mx[p[v]] = max(mx[p[v]], mx[v]);
    }
    for (int v = 1; v <= n; v++) cout << s[v] << ' ' << mx[v] << '\n';
    return 0;
}
```

ĐẾM SỐ PHÉP TÍNH

Lập order O(n), lượt ngược O(n). Chạy dưới 0,15 giây với $n = 2 \cdot 10^5$, chủ yếu là thời gian đọc dữ liệu.

Bảng điểm

Cách làm	Độ phức tạp	Subtask đạt	Điểm
Đi lên từ mỗi đỉnh	O(n · h)	1, 2	60
Gộp theo thứ tự BFS	O(n)	1, 2, 3	100

Lê Thành Nam